

Jaravi

El jefe determinista de tus agentes de IA

Orquestacion, gobierno y control de costos para sistemas multiagente

Documento de estrategia de producto y arquitectura

Version 0.3 — Julio 2026

Motor: .NET 8 / C# • Protocolo: Model Context Protocol (MCP) • Licencia propuesta: MIT (open-core)

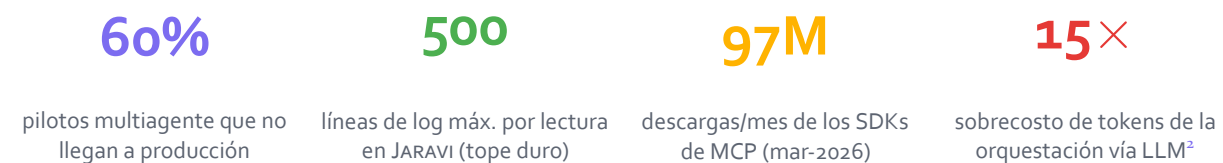
Contents

1	Resumen ejecutivo	2
2	El problema: colapso de contexto y explosión de costos	2
2.1	Colapso de contexto silencioso	2
2.2	Explosión de costos	2
2.3	La brecha de escalamiento	3
3	El mercado	3
3.1	Tamaño y crecimiento	3
3.2	MCP como estándar de facto	3
3.3	El ecosistema de CLIs que Jaravi ya orchestra	4
4	Arquitectura: motor determinista, no otro agente	4
5	Diferenciación técnica	5
5.1	Lo ya construido (vo.3, julio 2026)	5
6	Modelo de negocio: open-core	5
7	Roadmap	6
8	Conclusión	6

1 Resumen ejecutivo

JARAVI es un motor de orquestación *determinista* que convierte a un agente de IA de alto nivel (Claude Code, o cualquier cliente MCP) en el jefe de un equipo de sub-agentes externos — Open-Code, Codex CLI, Claude Code headless, GitHub Copilot CLI, Antigravity — sin que la conversación del jefe colapse por el volumen de logs, y sin pagar el sobre costo de tokens que implica usar un LLM para decidir cada paso de la orquestación.

La investigación de mercado 2026 confirma que este es exactamente el cuello de botella que estanca la adopción empresarial de sistemas multiagente: el **60% de los pilotos de IA multiagente no llegan a producción**, y la causa citada con más frecuencia es la **complejidad de orquestación**¹. JARAVI ataca ese cuello de botella con una decisión de arquitectura simple: el motor que decide cuándo lanzar, esperar, encolar o matar un sub-agente está escrito en C# determinista, no en un prompt. El LLM solo entra para decidir *qué* delegar — nunca para orquestar *cómo*.



2 El problema: colapso de contexto y explosión de costos

Tres investigaciones independientes de 2026 describen el mismo problema desde ángulos distintos, y los tres describen exactamente lo que JARAVI fue diseñado para resolver.

2.1 Colapso de contexto silencioso

En flujos de trabajo largos, el contexto acumulado llena la ventana del agente orquestador; cuando se acerca al límite, el modelo **empieza a descartar instrucciones anteriores en silencio** — incluyendo restricciones críticas³. Con cuatro o más trabajadores, el contexto excede la ventana con frecuencia. No es un fallo visible: el agente sigue respondiendo, pero con reglas que ya no recuerda.

El motor absorbe *todo* el output de los subprocesos en un ring buffer acotado por sesión. El jefe nunca recibe más de 500 líneas por lectura (tope duro server-side, configurable pero nunca desactivable), y el patrón recomendado es leer `get_summary` — un digest de exit code, duración y errores extraídos — en vez de logs crudos. Verificado en pruebas: una sesión que emitió 50,002 líneas devolvió exactamente 500 en la lectura.

2.2 Explosión de costos

Las herramientas agénticas cuestan entre \$200 y \$2,000+ por ingeniero al mes en gasto de tokens, y los sistemas multiagente multiplican ese costo: cada agente de un equipo de cinco consume tokens de forma independiente. Flujos que cuestan \$0.50 en pruebas pueden llegar a \$50,000/mes en producción a 100,000 ejecuciones, porque el orquestador hace múltiples llamadas al LLM solo para descomponer y agregar tareas, encima de cada llamada de los trabajadores⁴.

¹ Deloitte, 2025, citado en Viston.tech, “Multi-Agent Orchestration Challenges Enterprises Must Solve in 2026”.

² Estimado por Anthropic para arquitecturas multiagente orquestadas por LLM; citado en análisis de mercado 2026 sobre costos de sistemas agénticos. ³ FlowHunt, “Multi-Agent AI Systems in 2026: What the Research Actually Says”.

⁴ KGT Solutions, “Multi-Agent AI Orchestration: A CTO’s 2026 Guide”.

JARAVI nunca invoca un LLM para decidir el cómo de la orquestación — spawn, await, cola, kill son código C# determinista, con costo de cómputo, no de tokens. El jefe (el único LLM en el ciclo) paga tokens solo por decidir *qué* delegar y por leer resúmenes acotados. La tool run_agent colapsa el patrón spawn+await+summary en una sola llamada MCP, reduciendo también las rondas de ida y vuelta del jefe.

2.3 La brecha de escalamiento

El 80% de las empresas que empiezan con un solo agente planean orquestar múltiples agentes en dos años, pero **menos del 10% lo ha logrado**⁵. Sin coordinación, gobierno y observabilidad, los agentes duplican trabajo, producen resultados inconsistentes y crean puntos ciegos de seguridad.

Scope Gate valida cada workdir contra raíces permitidas antes de lanzar un proceso. Claims de rutas (vo.3) declaran qué sesiones pueden escribir dónde, con rechazo estructurado o cola FIFO automática — elimina la duplicación de trabajo por diseño, no por instrucción. El Dashboard WPF da observabilidad en tiempo real vía WebSocket sin tocar el contexto del jefe.

3 El mercado

3.1 Tamaño y crecimiento

Segmento	2024/25	2026	Proyección
Mercado de agentes de IA	\$5.25 mil M	—	\$52.62 mil M (2030, CAGR 46.3%)
Agentes autónomos	—	\$8.5 mil M	\$35 mil M (2030)
Asistentes de código IA	—	\$12.8 mil M	\$30.1 mil M (2032, CAGR 27%)
Servicios open-source	—	\$66.84 mil M	—

Table 1: Tamaño de mercado y proyecciones, fuentes de investigación de mercado 2026 (ver referencias).

Gartner proyecta que el 40% de las empresas integrará agentes de IA hacia fines de 2026, y los CIOs esperan que para 2030 el 75% del trabajo de TI involucre personas aumentadas con IA.

3.2 MCP como estándar de facto

El Model Context Protocol — el protocolo que JARAVI habla nativamente — ha cruzado de protocolo de nicho a infraestructura mainstream de agentes:

- **41%** de las organizaciones de software encuestadas están en producción limitada o amplia con servidores MCP.
- El registro oficial de MCP contaba **9,652** servidores únicos y **28,959** versiones al 24 de mayo de 2026.
- Los SDKs de TypeScript y Python de MCP alcanzaron **97 millones** de descargas mensuales en marzo de 2026.
- En diciembre de 2025, Anthropic donó MCP a la *Agentic AI Foundation* (AAIF), un fondo dirigido bajo la Linux Foundation cofundado con Block y OpenAI — consolidando a MCP como estándar neutral de proveedor.

⁵ Gartner, citado en Kanerika, “AI Agent Orchestration in 2026: What Enterprises Need to Know”.

- El panorama se ha estabilizado en un modelo de dos capas: **MCP** para integración vertical con herramientas, **A2A** (Google) para coordinación horizontal entre agentes.

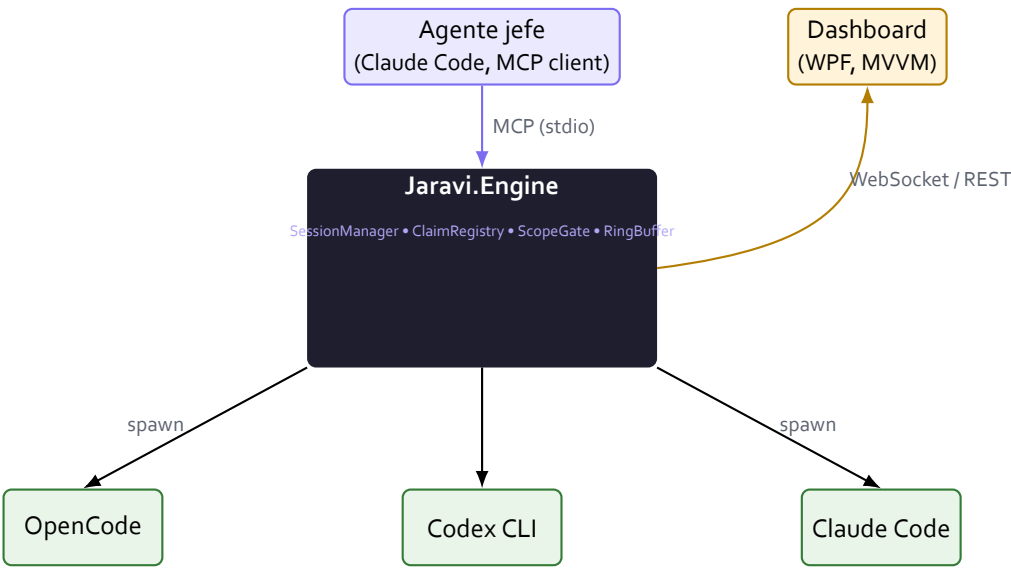
3.3 El ecosistema de CLIs que Jaravi ya orquesta

Agente	Posición de mercado (jul-2026)	Perfil en Jaravi
OpenCode	Líder open-source: 180,312 estrellas en GitHub (MIT)	opencode
Codex CLI (OpenAI)	#1 en Terminal-Bench 2.1, 83.4%	codex
Claude Code	46% de satisfacción (JetBrains, abr-2026), 134,868 estrellas	claude
GitHub Copilot	Líder en adopción: 29% uso laboral, 4.7M suscriptores	copilot
Cursor	\$2 mil M ARR, 1M+ usuarios de pago	(vía extensión de perfil)

Table 2: Cada fila es una entrada declarativa en `agents.json` — cero cambios de código para agregar un nuevo CLI.

4 Arquitectura: motor determinista, no otro agente

La decision de diseño central de JARAVI es que el **motor de orquestacion no es un agente de IA**. Es un programa .NET convencional con maquina de estados, colas y buffers — exactamente el tipo de sistema que la ingenieria de software sabe hacer confiable, auditable y barato de correr. El unico LLM en el ciclo es el jefe, que habla con el motor por MCP.



- **Jaravi.Core** — dominio puro: modelos, eventos, contratos. Cero dependencias externas.
- **Jaravi.Engine** — el motor determinista: `SessionManager` (ciclo de vida y máquina de estados), `ClaimRegistry` (bloqueos de rutas), `ChannelEventBus` (telemetría sin bloqueo), `RingBufferLogStore` (logs acotados), `ScopeGate` (seguridad de `workdir`).
- **Jaravi.McpServer** — host Kestrel: 10 tools MCP, WebSocket de eventos, REST. Distribuido como `dotnet tool` global (`jaravi-mcp`) — instalación de un comando, cero rutas absolutas en la configuración del cliente.
- **Jaravi.Dashboard** — observador WPF/MVVM, desacoplado del motor (solo consume HTTP/WebSocket). Nunca afecta al motor si se cierra.

5 Diferenciación técnica

Capacidad	Frameworks orquestados por LLM	Jaravi
Decisión de spawn/wait/kill	Inferencia del LLM en cada paso	Máquina de estados determinista en C#
Costo de orquestación	Tokens por cada decisión (“15× premium”)	Cómputo, no tokens
Colapso de contexto	Riesgo alto en sesiones largas	Ring buffer + tope duro de lectura (500 líneas)
Colisión de archivos entre agentes	Depende de la disciplina del prompt	Claims declarativos con cola FIFO o rechazo
Encadenar resultados entre agentes	El jefe copia y pega manualmente	inputFromSessionId — el motor inyecta el resultado
Agregar un nuevo CLI	Cambios de código / nuevo adaptador	Entrada declarativa en agents.json
Instalación	Script, Docker, o build manual	dotnet tool install -g — un comando

Table 3: Jaravi frente al patrón dominante de orquestación vía LLM.

5.1 Lo ya construido (v0.3, julio 2026)

- **10 tools MCP:** spawn_agent, run_agent (spawn+await+summary en una llamada), await_session, get_summary, read_output (acotado), send_input, kill_agent, list_agents, list_sessions, get_status.
- **Pipelines de sesiones:** encadena un agente auditor con un agente corrector sin que el jefe toque el resultado intermedio.
- **Claims de rutas:** paralelismo seguro entre sub-agentes que escriben archivos.
- **Distribución zero-friction:** dotnet tool global (jaravi-mcp), configuración de usuario auto-sembrada en %APPDATA%\jaravi.
- **54 pruebas automatizadas** (xUnit), incluyendo E2E contra procesos reales (timeout, kill, flood de 50k líneas, colas de claims).
- **5 perfiles de CLI reales** verificados en producción: OpenCode, Codex, Claude Code, Copilot, Antigravity.

6 Modelo de negocio: open-core

El patrón validado para herramientas de desarrollador en 2026 es *open-core*: núcleo libre que genera confianza, comunidad y distribución; capas comerciales para lo que las empresas exigen y los desarrolladores individuales no necesitan⁶. Según la encuesta 2023 de OpenLogic, el **67% de las empresas** que adoptan un producto open-core terminan pagando por el nivel superior.

Esta propuesta no es especulativa: la extensión *Enterprise-Managed Authorization* de MCP — ya en estado estable y adoptada por Asana, Atlassian, Canva, Figma, Linear y Supabase — es exactamente el tipo de necesidad de gobierno que el tier Enterprise de JARAVI resolvería para el motor de orquestación.

⁶ GetMonetizely, “Monetizing Open-Source Software: Pricing Strategies for Open-Core SaaS”; patrón validado por HashiCorp (Terraform) entre otros.

	Community (MIT, gratis)	Enterprise
Motor y tools MCP	Completo	Completo
Perfiles de agentes	agents.json local	Catálogo gestionado + plantillas verificadas
Dashboard	WPF de escritorio	Hosted, multi-usuario, multi-tenant
Gobierno	Scope Gate, claims locales	SSO/EMA, RBAC, políticas centralizadas
Auditoría	Eventos en memoria	Logs persistentes, exportación, retención
Soporte	Comunidad	SLA, cuenta dedicada

Table 4: Propuesta de tiers, alineada con la extensión de Autorización Gestionada por Empresa (EMA) que el propio ecosistema MCP está estandarizando en 2026.

7 Roadmap

1. **Ahora — vo.3:** pipelines, claims, distribución como dotnet tool, 5 perfiles de CLI reales.
2. **Corto plazo:** ConPtyIoStrategy (PTY real) para CLIs que exigen TTY interactivo; persistencia NDJSON de logs por sesión; métricas de costo/uso por sesión y por perfil.
3. **Mediano plazo:** Dashboard hosted multi-usuario; integración EMA para gobierno centralizado; publicación del paquete en NuGet.org.
4. **Largo plazo:** puente AzA para coordinación horizontal con agentes no-CLI; catálogo público de perfiles verificados por la comunidad.

8 Conclusión

El mercado de 2026 describe un problema preciso: las empresas quieren orquestar múltiples agentes de IA, pero el 60% de los intentos fracasa por complejidad de orquestación, colapso de contexto y costos que escalan de forma impredecible. JARAVI responde con una tesis de ingeniería simple y verificable: **la orquestación no necesita inteligencia artificial, necesita determinismo**. El LLM decide qué hacer; un motor .NET convencional decide cómo hacerlo — de forma auditable, barata y a prueba de colapso de contexto por diseño.

— fin del documento —

Referencias

- Viston.tech (2026). *AI Agent Orchestration in 2026: Enterprise Guide to Multi-Agent Systems; Multi-Agent Orchestration Challenges Enterprises Must Solve in 2026*.
- Kanerika (2026). *AI Agent Orchestration in 2026: What Enterprises Need to Know*.
- Deloitte (2025/2026). *Unlocking exponential value with AI agent orchestration*.
- KGT Solutions (2026). *Multi-Agent AI Orchestration: A CTO's 2026 Guide*.
- FlowHunt (2026). *Multi-Agent AI Systems in 2026: What the Research Actually Says*.
- Digital Applied (2026). *MCP Adoption Statistics 2026: Model Context Protocol*.
- InfoQ (2026). *AI Model Context Protocol Adds Centralised Auth for Enterprise*.
- Toloka AI (2026). *The future of MCP: 2026 roadmap, enterprise adoption, and what comes next*.
- Zylos Research (2026). *Agent Interoperability Protocols 2026: MCP, A2A, ACP and the Path to Convergence*.
- Morphllm (2026). *Best AI Coding Agent (2026): Ranked by Terminal-Bench, Price, and Source*.
- Ideaplan (2026). *AI Coding Assistant Market Share 2026: Cursor vs Copilot*.
- GetMonetizely (2026). *Monetizing Open-Source Software: Pricing Strategies for Open-Core SaaS*.